

VERIFORM

Verifiable AI Agents with TEE-Backed Decision Receipts

Proof, not blind trust.

Complete Technical Documentation

Version 0.1 — July 10, 2026

github.com/Sakthi-Sundaram-R/Veriform

Contents

1. Executive Summary
2. Problem Statement
3. Background: TEEs and Remote Attestation
4. Solution Approach
5. System Architecture
6. Component Reference
7. The Cryptographic Receipt
8. The Verification Pipeline
9. The Tamper Demonstration
10. The Judgment Layer (Pluggable AI)
11. Security Properties and Design Decisions
12. Live Test Results
13. Running Veriform
14. Roadmap and Current Status
15. API Reference
16. Configuration Reference

1. Executive Summary

Veriform makes AI agents provable. Every decision an agent makes is wrapped in a **cryptographic receipt** that binds that exact decision to unaltered code running inside a hardware-secured enclave (a Trusted Execution Environment, or TEE). Anyone — an end user, an auditor, or a smart contract — can verify the receipt in seconds and reject any output that was tampered with, forged, or produced outside the enclave.

The reference application is a **wallet-guardian agent**: a service that approves or denies crypto transfer requests using hard safety rules plus an LLM judgment layer. The decision engine is deliberately simple, because the decision is not the product — **the receipt is**. The same receipt mechanism generalizes to any agent whose outputs carry consequences: financial agents, medical triage, compliance audits, autonomous on-chain actors.

The system is fully implemented and demonstrated end to end: an honest agent whose receipts verify (including LLM-reasoned verdicts), and two attacker variants — an impostor with no attestation and a forger with a fabricated quote — both rejected by the verifier for real cryptographic reasons, not hardcoded checks.

2. Problem Statement

2.1 The trust gap

AI agents increasingly act on their own: approving transactions, handling private user data, executing on-chain operations without a human in the loop. But every agent runs on infrastructure that someone controls. Whoever operates the server can read the agent's inputs, alter its decisions, swap its code, or extract the private keys and API secrets it holds — and the end user has no way to detect any of it.

When an agent signs a transaction or makes a financial decision, there is normally no way to prove that:

- the agent ran the exact code it claims to run, unmodified;
- its decision was genuinely produced inside a secure environment, not forged or manipulated by the host;
- its private keys never left that environment or were exposed to the operator.

2.2 The primitive is solved; the gap is above it

Running agents inside TEEs is no longer theoretical. Platforms such as Phala, Marlin and Atoma, and research systems such as VET and Attestable Audits, have proven the core primitive: an agent can run in a hardware enclave and produce a **remote attestation** that cryptographically proves its code is unaltered and its secrets stayed sealed. What remains unsolved is everything *between that primitive and a person or contract who needs to trust it*:

- **Attestation is unusable by the people who need it.** A remote attestation is a raw cryptographic quote. A user approving a transaction, or a smart contract gating an action, has no simple, fast, trustworthy way to consume it. There is no clean 'verify in ten seconds' layer.
- **Implementations are quietly insecure.** Analyses of open-source TEE projects have found large fractions bypassing official SDK cryptographic APIs or exhibiting insecure coding practices. The hardware guarantee is strong; the software wrapping it often silently breaks it.

- **Trust rests on a single vendor.** Nearly all attested inference depends on one vendor's signing infrastructure. A single PKI compromise breaks the whole guarantee.

2.3 The challenge

Build the verification layer that makes an attested agent's guarantee **consumable**: a system where any third party, human or smart contract, can receive an agent's signed decision and confirm — in seconds, without trusting the operator — that it came from a genuine, unaltered enclave. Make the invisible proof visible, correct, and hard to fake.

Success criterion: a non-expert user (or an on-chain contract) receives an agent's decision, sees a clear verified/not-verified result derived from a real attestation, and reliably rejects any output that was tampered with, forged, or produced outside the enclave.

3. Background: TEEs and Remote Attestation

3.1 Trusted Execution Environments

A TEE is a hardware-isolated region of a CPU (Intel SGX/TDX, AMD SEV) in which code and data are encrypted and protected even from the machine's own operating system, hypervisor, and root user. Code inside the enclave can hold secrets — signing keys, API credentials — that the host physically cannot read.

3.2 Remote attestation

On demand, the CPU produces a **quote**: a data structure signed by the hardware vendor's key hierarchy containing a *measurement* (a hash of the exact code loaded into the enclave) and a caller-supplied field called **report_data** (64 bytes). A verifier can check the quote's signature chain against the vendor's public key infrastructure and compare the measurement to a known-good build. If anyone swapped the code, the measurement changes and verification fails.

The `report_data` field is the crucial extension point: because the hardware signs whatever the enclave code places there, an application can bind arbitrary claims — such as 'I made this exact decision' — to the attestation. Veriform is built on precisely this trick.

3.3 Phala dstack

Veriform uses Phala Network's **dstack** runtime, which provides full-VM TEE isolation: an ordinary Docker container is deployed unmodified into an Intel TDX confidential VM. Inside the container, a Unix socket (`/var/run/dstack.sock`) exposes the guest agent API through which the application requests quotes and derives sealed keys. A local simulator provides the same API for development without TEE hardware, so the identical container image moves from laptop to real silicon with zero code changes.

4. Solution Approach

4.1 The core idea: bind each decision to the enclave

Attestation normally proves 'this code is running unaltered.' Veriform extends it to prove '**this specific decision** was made by that unaltered code.' The binding scheme:

```
canonical      = deterministic JSON of the decision payload
                  (sorted keys, no whitespace)
decision_hash  = SHA-256(canonical)
binding        = SHA-256(decision_hash || lowercase(agent_address))
report_data    = binding padded to 64 bytes

quote          = hardware attestation with report_data inside
signature      = secp256k1 signature over canonical,
                  by a key derived INSIDE the enclave
```

Because `report_data` is covered by the hardware signature, and because it commits to both the decision hash and the enclave-held signing key, an attacker cannot:

- alter the decision after the fact (`decision_hash` changes, binding breaks);
- replay a legitimate quote for a different decision (same reason);
- sign with a key outside the enclave (the binding commits to the attested key's address, and the key itself is derived inside the TEE and never leaves it).

4.2 The receipt

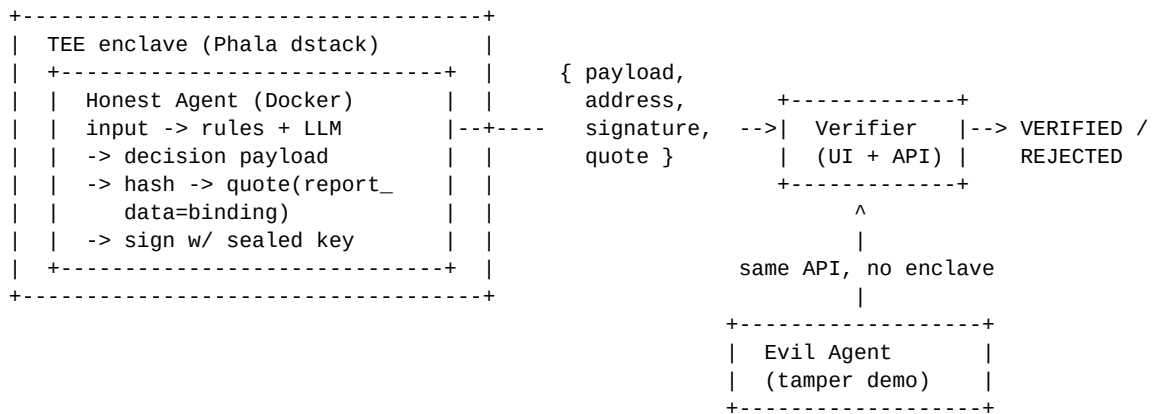
Every agent response carries four elements:

Field	Content
payload	The decision itself: the request, APPROVE/DENY, the method (rules or LLM), reasoning notes, timestamp, agent version
address	Ethereum-style address of the enclave-derived secp256k1 key
signature	Signature over the canonical payload by that key
quote	The TDX attestation quote whose report_data binds the decision

4.3 The verifier

The verifier runs *outside* the enclave and trusts nothing but mathematics. Given a receipt, it recomputes the canonical payload, the decision hash and the expected binding, extracts report_data from the quote at the TDX-specified byte offset, recovers the signer from the signature, and — on real hardware — validates the quote's signature chain against Intel's PKI. The result is rendered as a single, unambiguous verdict (VERIFIED / REJECTED) with a per-check breakdown, consumable by a non-expert in seconds.

5. System Architecture



Four services compose the demo (docker-compose or bare uvicorn):

Service	Port	Role
agent	8001	Honest agent inside the enclave/simulator; makes decisions and emits receipts
evil-agent	8002	Attacker: identical API, approves everything, signs with a non-enclave key; sends no quote or a forged one
verifier	3000	Verification API + browser UI with the three-way demo toggle
dev-sim	8090	Development shim speaking the dstack wire protocol on platforms without the official simulator (clearly labeled NOT-A-TEE)

6. Component Reference

6.1 agent/ — the honest agent

- **app/main.py** — FastAPI service. POST /decide accepts a transfer request, obtains a verdict from the decision engine, assembles the payload, and asks the enclave module to attest it. If the enclave is unreachable it returns 503 rather than an unattested answer.
- **app/decision.py** — the decision engine. Hard rules first (address validity, positive amount, hard limit, trusted-address auto-approval, mandatory reason), then a pluggable LLM judgment layer for the gray zone. Every path fails closed (Section 10, 11).
- **app/enclave.py** — everything that touches the TEE: derives the secp256k1 account from a dstack-derived key, computes the binding, requests the quote, signs the canonical payload.

6.2 evil-agent/ — the attacker

Impersonates the honest agent: same API, same response shape, same claimed agent version. It is 'compromised' — it approves every transfer. It signs with a freshly minted key that never saw an enclave. Two modes, selectable per request (?mode=forged) or by environment variable: **none** sends no quote at all; **forged** sends a structurally plausible quote of the right length filled with garbage whose report_data binds to nothing.

6.3 verifier/ — the verification layer

- **app/verify.py** — the five-check pipeline (Section 8), including TDX quote parsing at the specified offsets.
- **app/main.py** — POST /ask proxies a request to the selected agent, verifies the returned receipt, and returns both. Serves the UI.
- **static/index.html** — single-page UI: three-way agent toggle, transfer form, giant VERIFIED/REJECTED verdict, per-check breakdown.

6.4 dev-sim/ — the development shim

The official Phala simulator does not yet run on Windows. dev-sim/sim.py implements the two dstack guest-agent RPCs the agent uses (POST /GetKey, POST /GetQuote) so the unmodified agent runs anywhere. Like the official simulator, its quotes are structurally valid TDX quotes with the caller's report_data at the correct offset but carry no hardware attestation chain — the quote header literally reads 'DSTACK-DEV-SHIM:NOT-A-REAL-QUOTE', and the verifier honestly reports the authenticity check as skipped in this mode.

7. The Cryptographic Receipt

A real receipt produced during testing (quote truncated):

```
{
  "payload": {
    "agent": "veriform-agent/0.1",
    "request": {
      "to": "0x8ba1f109551bD432803012645Ac136ddd64DBA72",
      "amount": 2.5, "token": "ETH",
      "reason": "monthly rent payment to landlord as agreed"
    },
    "action": "APPROVE",
    "method": "llm (gemini-flash-lite-latest)",
    "notes": "Consistent with a standard recurring payment; no signs
      of urgency or malicious patterns.",
    "timestamp": "2026-07-10T04:30:11.412208+00:00"
  },
  "address": "0x3d5fcC7673F53075F26A6944B84c4eF8B49A0e79",
  "signature": "dd836c19...079d1c",
  "quote": "44535441434b2d4445562d5348494d..." (632 bytes)
}
```

7.1 Canonicalization

Verifier and agent must hash byte-identical input, so the payload is serialized with sorted keys and no whitespace (`json.dumps(payload, sort_keys=True, separators=(',', ':'))`). Any post-hoc modification of any field — flipping DENY to APPROVE, editing the amount, backdating the timestamp — changes the canonical bytes, which changes the decision hash, which breaks both the quote binding and the signature simultaneously.

7.2 Quote layout

TDX quote v4: a 48-byte header followed by a 584-byte TD report body. `report_data` occupies the final 64 bytes of the body, i.e. absolute bytes 568–632. The verifier extracts exactly that slice and compares its first 32 bytes to the recomputed binding.

8. The Verification Pipeline

Five checks run in order; any hard failure yields REJECTED:

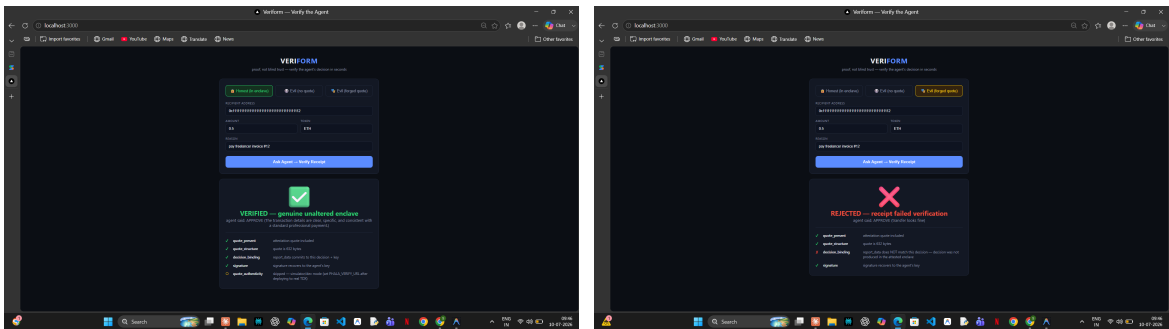
#	Check	Question it answers	Catches
1	quote_present	Did the response include an attestation quote at all?	Impostors running outside any enclave
2	quote_structure	Does the quote parse as hex and meet the minimum TDX quote size (632 bytes)?	Malformed or truncated quotes
3	decision_binding	Does <code>report_data</code> inside the quote equal <code>SHA-256(decision_hash address)</code> ?	Forged quotes, replayed quotes, altered decisions
4	signature	Does the signature over the canonical payload recover to the claimed address?	Tampered payloads, wrong signers
5	quote_authenticity	Does the quote's signature chain validate against Intel's PKI?	Entirely fabricated quotes (real TDX deployments only; reported honestly as 'skipped' in simulator/dev mode)

There is no 'if agent is evil' logic anywhere. Rejections emerge from the mathematics: an attacker either lacks a quote, or presents one whose `report_data` cannot commit to the decision it is trying to sell.

9. The Tamper Demonstration

The demo's purpose is to make invisible security tangible. The UI offers three modes against the same transfer form:

Mode	What it is	Verifier outcome
Honest (in enclave)	Real agent, enclave-derived key, genuine binding	VERIFIED — all checks pass
Evil (no quote)	Impostor outside the enclave; valid signature but no attestation	REJECTED — quote_present fails
Evil (forged quote)	Same impostor, now attaching a structurally valid fake quote	REJECTED — decision_binding fails, even though quote presence, structure and signature all pass



Left: honest agent VERIFIED. Right: forged quote REJECTED at decision_binding — the receipt passes every superficial check and is still caught.

The forged-quote case is the decisive one. A skeptic can see that the attacker produced a receipt that is superficially perfect — correct length, valid hex, a signature that genuinely recovers to the attacker's key — and the verifier still rejects it, because only code running inside the attested enclave can get the hardware to sign a report_data that commits to this decision and this key.

10. The Judgment Layer (Pluggable AI)

Gray-zone decisions — transfers that pass the hard rules but are not auto-approvable — are delegated to an LLM. The provider is pluggable precisely because the receipt does not care which brain made the decision; the enclave attests the decision process wholesale.

Provider	Backend	Cost	Notes
anthropic	Claude (claude-opus-4-8) via the Anthropic API with structured JSON output	paid	Highest quality; one env var to enable
gemini	Google Gemini free tier via REST (gemini-flash-lite-latest)	free	Current default in the live demo; JSON response mode, output-token headroom, backoff retry for rate limits
ollama	Local model via Ollama (llama3.2:3b), schema-constrained decoding	free	Fully offline; for machines with the model installed
none	—	free	Rules only; gray zone denied conservatively

Provider resolution: an explicit JUDGE_PROVIDER wins; otherwise *auto* picks anthropic if ANTHROPIC_API_KEY is set, then gemini if GEMINI_API_KEY is set, then ollama. **Every provider fails closed:** an API outage, a bad key, a malformed model response, or a safety refusal results in DENY — never a silent approval — and the failure reason is recorded in the signed receipt itself.

11. Security Properties and Design Decisions

11.1 Fail closed, provably

A wallet guardian whose judgment layer is down must not approve transfers. Veriform encodes this: any exception in the judgment path produces DENY with the error class in the notes — and because the notes are part of the signed payload, even the failure mode is auditable. This property was exercised repeatedly in live testing (API billing errors, model output truncation, free-tier rate limits): every failure surfaced as a verified DENY.

11.2 Keys never exist outside the enclave

The signing key is derived inside the TEE via the dstack key-derivation API (`get_key`), scoped to an application path. It is never written to disk, never present in the container image, and never transmitted. The host operator cannot extract it; a leaked container image contains nothing to steal.

11.3 The verifier trusts only mathematics

The verifier shares no secrets with the agent. Everything it checks is recomputable from the receipt plus public knowledge (hash functions, signature recovery, the TDX quote layout, Intel's public PKI). It can therefore run anywhere — a browser, a backend, or in reduced form an on-chain contract.

11.4 Honest about simulation

In development the quotes come from a simulator and carry no hardware chain. Veriform does not paper over this: the authenticity check reports 'skipped — simulator/dev mode' rather than a fake pass, and the dev shim labels its quotes NOT-A-REAL-QUOTE in the header bytes. On Phala Cloud deployment, the same check turns into a hard pass/fail against Intel's PKI.

11.5 Known limitations (current build)

- Quote authenticity is only enforceable on real TDX hardware (Phase 3 of the roadmap); locally it is transparently skipped.
- The verifier does not yet pin an expected enclave measurement (MRTD) or the agent's compose hash; production use would compare these against a published known-good build.
- A malicious operator can still refuse service (liveness is out of scope); what they cannot do is forge or alter decisions undetected.

12. Live Test Results

12.1 Cryptographic unit scenarios (local)

Scenario	Expected	Result
Honest receipt	VERIFIED	PASS
Payload tampered after signing (DENY flipped to APPROVE)	REJECTED: binding + signature	PASS
Missing quote	REJECTED: quote_present	PASS

Scenario	Expected	Result
Forged quote (garbage bytes)	REJECTED: decision_binding	PASS
Signature from wrong key	REJECTED: signature	PASS

12.2 End-to-end demo (full stack, live LLM)

Act	Input	Outcome
Honest + legit	2.5 ETH, 'monthly rent payment to landlord as agreed'	APPROVE via llm; receipt VERIFIED
Honest + scam	4.5 ETH, 'URGENT limited giveaway - send now to double your ETH back guaranteed'	DENY via llm ('classic scam pattern'); receipt VERIFIED — a provable denial
Evil impostor	Same rent transfer	Claims APPROVE; REJECTED at quote_present
Evil forger	Same rent transfer	Claims APPROVE; quote, structure and signature all pass; REJECTED at decision_binding

Stability: six consecutive LLM-judged requests completed 5/6 with the sixth failing closed under a free-tier rate limit (429) — i.e. the failure mode itself behaved as designed. After switching to the higher-quota flash-lite model and adding backoff, runs completed cleanly.

13. Running Veriform

13.1 Linux / macOS (official simulator)

```
npm install -g phala
phala simulator start          # local TEE simulator, no hardware
docker compose up
# open http://localhost:3000
```

13.2 Windows / no Docker (dev shim)

```
pip install fastapi uvicorn httpx eth-account dstack-sdk anthropic

# four terminals (or background jobs):
uvicorn sim:app --port 8090 --app-dir dev-sim
set DSTACK_SIMULATOR_ENDPOINT=http://localhost:8090
uvicorn app.main:app --port 8001 --app-dir agent
uvicorn app.main:app --port 8002 --app-dir evil-agent
uvicorn app.main:app --port 3000 --app-dir verifier
```

13.3 Real TEE deployment (Phala Cloud)

```
phala auth login
phala deploy -c docker-compose.yaml -n veriform-agent
phala cvms attestation <cvm-id> # genuine Intel TDX quote
# point the verifier at the deployed URL and set PHALA_VERIFY_URL
```

The same container image runs in all three environments; only the source of quotes changes. On real TDX the fifth verification check (quote_authenticity) becomes an enforced pass/fail against Intel's PKI.

14. Roadmap and Current Status

Phase	Scope	Status
Build	Agent, evil agent, verifier + UI, docker-compose, dev shim; live end-to-end run	DONE
Phase 2	LLM judgment live: pluggable provider (anthropic/gemini/ollama/none), all fail closed; verified on the Gemini free tier	DONE
Phase 4	Demo polish: three-way tamper toggle, 2-minute demo script (DEMO.md), README screenshots	DONE
Phase 1	Run against the official Phala simulator (requires Docker or WSL)	PENDING
Phase 3	Deploy to real Intel TDX on Phala Cloud; enable PHALA_VERIFY_URL so all five checks pass on genuine silicon	PENDING
Phase 5*	On-chain anchor: testnet contract storing the attested public key; contract action gated on 'signed by a verified enclave'	STRETCH
Phase 6*	Multi-vendor attestation: require agreement across Intel/AMD/Nvidia roots	STRETCH

15. API Reference

Agent (honest and evil) — ports 8001 / 8002

```

GET /health
-> {"status": "ok", "agent": "veriform-agent/0.1"}

POST /decide
body: {"to": "0x...", "amount": 2.5, "token": "ETH",
      "reason": "..."}
evil agent only: ?mode=forged (per-request forged quote)
-> {"payload": {...}, "address": "0x...",
   "signature": "...", "quote": "..." | null}
503 if the enclave is unreachable (honest agent)

```

Verifier — port 3000

```

GET / -> demo UI

POST /ask
body: {"agent": "honest" | "evil" | "evil-forged",
      "to": "0x...", "amount": 2.5, "token": "ETH",
      "reason": "..."}
-> {"agent": ..., "receipt": {...},
   "verification": {"verified": bool,
                   "verdict": "VERIFIED"|"REJECTED",
                   "checks": [{name, passed, detail}, ...]}}

```

Dev shim — port 8090 (dstack wire protocol)

```

POST /GetKey {path, purpose, algorithm}
-> {key: <32-byte hex>, signature_chain: []}
POST /GetQuote {report_data: <hex>}
-> {quote: <hex>, event_log: ..., report_data: <hex>}

```

16. Configuration Reference

Variable	Default	Purpose
JUDGE_PROVIDER	auto	anthropic gemini ollama none auto
ANTHROPIC_API_KEY	—	Enables the Claude judgment provider
ANTHROPIC_MODEL	claude-opus-4-8	Claude model id
GEMINI_API_KEY	—	Enables the Gemini free-tier provider
GEMINI_MODEL	gemini-flash-lite-latest	Gemini model id
OLLAMA_URL	http://localhost:11434	Local Ollama endpoint
OLLAMA_MODEL	llama3.2:3b	Local model name
MAX_TX_AMOUNT	5.0	Hard limit; larger transfers always denied
TRUSTED_ADDRESSES	(example addr)	Comma-separated auto-approve allowlist
TRUSTED_AUTO_LIMIT	1.0	Max auto-approved amount for trusted addresses
DSTACK_SIMULATOR_END POINT	—	HTTP endpoint of the simulator/shim (otherwise the Unix socket is used)
EVIL_MODE	none	Evil agent default: none forged

Variable	Default	Purpose
PHALA_VERIFY_URL	—	Quote-authenticity endpoint; set after deploying to real TDX

— End of document —